



UNIVERSITÉ PARIS-SACLAY

FACULTÉ DES SCIENCES

Rapport de projet

Jumeau numérique

M1 E3A

Rédigé par :
Ivan ZABOROWSKI
20220730

5 mai 2026

Objectifs

Ce document a pour objectif de décrire :

- L'installation de ROS 2 FOXY sur Ubuntu 20.04.6 LTS ;
- Le script de collecte de données avec ROS 2 ;
- Le script de machine learning (neural ODE) ;

1 Pré-requis

- Ubuntu 20.04.6 LTS ;
- Accès Internet ;
- Accès à un GPU NVIDIA (optionel) ;
- Fichier projet ou lien GitHub : <https://github.com/IvanZaborowski/Digital-Twin> ;

2 Installation de ROS 2 Foxy (Ubuntu)

2.1 Configuration locales

```
sudo apt update
sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
sudo apt install -y curl gnupg2 lsb-release
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master
    /ros.asc | sudo aptkey add -
sudo sh -c 'echo "deb http://packages.ros.org/ros2/ubuntu
    $(lsb_release -cs) main" > /etc
    /apt/sources.list.d/ros2-latest.list'
sudo apt update
```

2.2 Installation de ROS 2 Foxy

```
sudo apt install -y \
    ros-foxy-desktop \
    python3-rosdep python3-colcon-common-extensions python3-argcomplete
# rosdep ( faire une fois)
sudo rosdep init
rosdep update
```

2.3 Activation de l'environnement ROS

```
echo "source /opt/ros/foxy/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

3 Clonage du dépôt et compilation

3.1 Création du workspace

```
mkdir -p ~/tello_ros_ws/src  
cd ~/tello_ros_ws/src
```

3.2 Clonage des dépendances

```
git clone https://github.com/clydemcqueen/tello_ros.git  
git clone https://github.com/ptrmu/ros2_shared.git
```

3.3 Installation des dépendances via Rosdep et compilation

```
cd ~/tello_ros_ws  
rosdep install --from-paths src -y --ignore-src  
colcon build --event-handlers console_direct+  
source install/setup.bash
```

4 Connexion au drone

- Allez sur un serveur Ubuntu ;
- Connectez l'antenne WiFi au PC ;
- Mettez une batterie dans le drone et chargez les autres dans le hub ;
- Restez appuyer sur le bouton d'alimentation du drone pour l'allumer ;
- Connectez le PC au drone via WiFi ;

5 Vérification du code ROS 2

5.1 Récupération du code ROS 2

Le code est disponible via le lien GitHub ou directement dans le fichier projet.

5.2 Vérification de la communication PC-Drone

Dans la section "Mission" du code, commentez les paragraphes de décollage, d'atterrissage et des différentes phases.

Ensuite lancez le code, le drone doit afficher :

```
Commande envoyée : command  
Commande reçue : ok  
Batterie : 85 % Par exemple  
Command : ok
```

6 Missions et collecte des données

6.1 Test d'un décollage et atterrissage

Décommentez, dans la section "Mission", les paragraphes de décollage et d'atterrissage et laissez commenté les différentes phases.

Ensuite lancez le code, le drone doit décoller pendant 3s et atterrir.

Un dataset est créé dans `dataset_neural_ode\drone_data.csv`. Vérifiez que les données de vitesses, d'accélération et d'inclinaisons sont correctement récupérées.

6.2 Test d'une commande de base

Décommentez, dans la section "Mission", une des phases et vérifiez le résultat sur le drone et dans le dataset. Si le dataset n'est pas actualisé à l'issue du code, supprimez le fichier `.csv` et relancez le code.

Préparez des phases permettant le plus de mouvement en même temps pour avoir un dataset riche lors de la collecte réelle de données.

6.3 Vérifications du topic VICON

Lors de la séance VICON, vérifiez, dans un terminal, le topic associé à VICON avec :

```
ros2 topic list
```

Modifiez `/vicon/drone/pose` de la section "Subscriber VICON" avec le nom réel du topic.

7 Récupération des datasets simulés et du script de machine learning

7.1 Récupérations des datasets

Récupérez les datasets simulés de Gazebo via le lien GitHub ou le fichier projet.

7.2 Récupération du script de machine learning

Récupérez le script de machine learning via le lien GitHub ou le fichier projet.

7.3 Ouverture de Jupyter Notebook

Ouvrez Anaconda puis Jupyter Notebook via l'application ou ouvrez un prompt anaconda et écrivez :

```
jupyter notebook
```

Ensuite ouvrez le fichier contenant le script avec Jupyter Notebook.

8 Réalisation du digital twin

8.1 Test avec datasets simulés

Modifiez les deux lignes contenant le chemin vers des datasets et mettez à la place le chemin vers le dataset `run_20260126_124602_circle_xy`.

Lancez la première partie et attendez le message "Installation 1 terminé", cela peut prendre 2 à 3min.

Lancez la seconde partie et attendez le message "Installation 2 terminé", cela devrait prendre que quelques secondes.

Lancez la troisième partie et vérifiez que vous avez une version compatible avec cuda, par exemple : 2.7.1+cu118. Sinon cela indique que vous utilisez un gpu non compatible (donc pas Nvidia), ce qui risque de ralentir le processus d'apprentissage, prenant environ 25min par test contre 5min avec le gpu Nvidia.

Lancez la quatrième partie pour confirmer l'utilisation du cuda si vous avez un gpu Nvidia.

Lancez la dernière partie qui devrait prendre 5 ou 20min selon le gpu. Vous devriez avoir des courbes comparant réel vs prédit ainsi que d'autres courbes et graphiques.

8.2 Test avec dataest réel

Remplacez les deux lignes contenant le chemin vers le dataset de Gazebo par le votre (obtenue avec VICON et ROS 2).

Relancez la dernière partie du code uniquement, observez les courbes d'erreurs, et les différentes courbes prédites et modifiez les hyperparamètres en fonction de l'erreur à corriger.